



// Master in Computer Engineering

ICCBD – Compact Summary

Andrea Sabbioni, Antonio Corradi

by Matteo Fontolan



// 2025/2026

Disclaimer & Info

This document is a personal collection based on lectures and/or educational materials provided by professors and/or collected over the years by students.



Attention

The author assumes no responsibility for any errors, omissions, or inaccuracies. The material is provided “as is” for educational support purposes only.

If you want to **add** useful material or **report** errors, please do so [here](#).

I hope this resource proves useful to you.

Good study and good luck! 🍀

If you'd like to support me with a chocolate 🍫, you can do so [here](#).

[Bento Profile](#)

Table of Contents

1. FOUNDATIONS	6
1.1. Cloud & Data Centers	6
1.2. Distributed Systems Basics	6
1.3. Resource Management & Scalability	7
1.4. UNIX Files & Atomicity	7
2. MIDDLEWARE & ARCHITECTURE	8
2.1. Containers & Microservices	8
2.2. Middleware Taxonomy & Cloud Models	8
2.3. Cloud Strategies: CAP, ACID, BASE	9
2.4. CORBA, Pub/Sub & MOM	9
3. CLOUD INFRASTRUCTURE	11
3.1. Kubernetes	11
3.2. OpenStack	12
3.3. Serverless & FaaS	12
3.4. Stream Processing	13
4. DISTRIBUTED COORDINATION	14
4.1. Replication for Dependability	14
4.2. Group Communication & Multicast Ordering	15
4.3. Synchronization & Logical Clocks	15
4.4. Mutual Exclusion Algorithms	16
4.5. Election Protocols	16
4.6. Global State & Distributed Snapshot	16
5. NETWORK QUALITY OF SERVICE	18
5.1. QoS Indicators & Management	18
5.2. IntServ vs DiffServ	18
5.3. Traffic Shaping & Scheduling	18
6. BIG DATA INFRASTRUCTURE	20
6.1. Overlay Networks & Distributed File Systems	20
6.2. NoSQL: Cassandra & MongoDB	20
6.3. MapReduce, Hadoop YARN & Apache Spark	21





1. FOUNDATIONS

1.1. Cloud & Data Centers



Cloud

A **coordinated network of geographically distributed data centers** providing unified, high-quality services. DCs are far apart enough to survive disasters independently and offer geographically replicated services.

DC Traffic:

- **East-West** – server↔server within DC (now dominant: replicas, microservices)
- **North-South** – DC ↔ external world (passes through core/border switches)

Big Data – 5 (6) Vs: Volume · Velocity · Variety · Veracity · Value (· *Variability*)

Topology	Key property
3-Tier	Core / Aggr / Access – simple, bottleneck at core
Leaf-Spine	Any leaf→spine in 1 hop – uniform latency
Fat-Tree / Clos	More links upward – bandwidth, redundancy, non-blocking

1.2. Distributed Systems Basics



Resource

Any component (HW/SW) needed during execution to produce a visible result in a distributed system.



Service

Abstraction of a business process with a **standard published API**, reusable, **black-box**.



Component

Static entity with a defined **port-based interface**. Hosted inside a **Container** (server-side runtime environment).

Transparency vs Visibility:

- **Transparency** – user sees one unified system (replication, location, failure hidden)
- **Visibility** – system exposes internal state for observability and control (dual goal)

Monitoring vs Observability:

- **Monitoring**: collect current metrics (reactive, what is happening)
- **Observability**: analyze + understand **why** + adapt (proactive)

TINA-C: 3-view architecture: **User, Provider, Vendor** over a Distributed Processing Environment (DPE).



SOA – 3 Actors



Provider publishes interface to **Registry**. **Consumer** discovers and binds. Contract = interface, not implementation. Granularity problem: too fine = chatty; too coarse = inflexible.

Microservice: fine-grained, independently deployable service implementing ONE business function.
Monolith → single deployable unit, simple but hard to scale parts independently.

1.3. Resource Management & Scalability



SLA – Service Level Agreement

Formal **non-ambiguous contract** between provider and consumer specifying QoS parameters (latency, availability, throughput) and penalties for violations.

Process Migration:

- **Sender initiative**: loaded node pushes tasks out
- **Receiver initiative**: idle node pulls tasks in
- **Internal problems**: open files, sockets, shared memory
- **External problems**: network topology, remote references

Farm Pattern (Master/Workers): 1 Master distributes tasks to N Workers in parallel. Classic for embarrassingly parallel workloads.

Amdahl's Law: $\text{Speedup}(N, p) = \frac{1}{1 - p + \frac{p}{N}}$ where p = parallelizable fraction, N = processors. The serial fraction $(1 - p)$ is the ultimate bottleneck.

IaC – Infrastructure as Code: manage and provision infrastructure using **version-controlled config files** (Terraform, Ansible). Reproducibility + automation.

CI/CD: Continuous Integration + Continuous Delivery – automated build, test, deploy pipeline enabling rapid iteration.

1.4. UNIX Files & Atomicity



UNIX Per-Primitive Atomicity

A UNIX **primitive** is **atomic** – the kernel applies its effects as one uninterruptible action. No **multi-step operation** is **atomic** by default – concurrent interleavings are possible.

i-node: metadata structure tracking all disk blocks of a file. **Directory**: special file mapping names → i-nodes. Concurrent writes to the same file produce non-deterministic results; directory listing under concurrent modification may miss or duplicate entries (**eventual consistency at primitive level**).



2. MIDDLEWARE & ARCHITECTURE

2.1. Containers & Microservices

Linux Namespaces (isolation scope per-process): Container vs VM:

- MNT: filesystem mounts
- PID: process ID space
- NET: network stack (interfaces, routes)
- IPC: shared memory / semaphores
- UTS: hostname / domain name
- USR: user and group IDs
- CGRP: cgroup root

	Container	VM
Kernel	Shared	Own
Startup	ms	seconds
Overhead	Minimal	Full OS
Isolation	Namespace+cgroup	Hypervisor

Linux cgroups: resource limiting, accounting, and isolation for process groups – cpu, memory, blkio, net_cls, cpuset.

Docker Image: read-only layered filesystem snapshot. Each Dockerfile instruction adds a layer (Copy-on-Write). Running container adds a thin writable layer on top.



Circuit Breaker

Stability pattern preventing cascading failures. States: CLOSED (normal) → OPEN (fail-fast, no calls) → HALF-OPEN (probe). Thresholds: failure rate + call volume + slow call rate.

2.2. Middleware Taxonomy & Cloud Models

Type	Description	Example
RPC/RMI	Remote invocation as local call (stub/skeleton)	gRPC, Java RMI
MOM	Async message exchange, decoupled in time/space	Kafka, RabbitMQ
DOC/OO	Distributed object calls via ORB	CORBA
DTP Monitor	Distributed ACID transactions	JTA, CICS, Tuxedo
DB Middleware	Heterogeneous DB integration	ODBC, JDBC
Self-*	Autonomous adaptation (self-heal, self-configure)	Autonomic MW

Service Model	What the provider manages	User manages
SaaS	Everything (infra, platform, app)	Nothing – just uses the app
PaaS	Infra + OS + runtime	App code + data
IaaS	Physical HW + virtualization	OS, runtime, app, data
FaaS	Infra + OS + runtime + scaling	Function code only



2.3. Cloud Strategies: CAP, ACID, BASE



CAP Theorem

At most 2 of 3 can be simultaneously guaranteed:

- **C – Consistency:** all nodes see the same data at the same time
- **A – Availability:** every request receives a response (possibly stale)
- **P – Partition Tolerance:** works despite network partitions

In practice P is mandatory in any distributed system → trade-off is CP vs AP.

- **CP:** ZooKeeper, HBase, MongoDB (primary) – consistent, may be unavailable during partition
- **AP:** Cassandra, DynamoDB, CouchDB – always available, may serve stale data

ACID (maximum consistency):

- **Atomicity:** all-or-nothing
- **Consistency:** DB rules always preserved
- **Isolation:** concurrent = serial execution
- **Durability:** committed = permanent

Two-Phase Commit (2PC):

1. Coordinator → all: **Prepare** (vote yes/no)
2. If all yes → **Commit**; else → **Abort**

Blocking if coordinator fails after Prepare.

BASE (“opposite” of ACID for cloud):

- **Basically Available:** system responds (possibly stale)
- **Soft state:** data may be in flux across replicas
- **Eventual Consistency:** converges when writes stop

Eventual Consistency: if all writes stop to a key, all replicas will converge to the same value.

eBay 5 Commandments: Partition Everything · Async Everywhere · Automate Everything · Everything Fails · Embrace Inconsistency

2.4. CORBA, Pub/Sub & MOM



CORBA

Common Object Request Broker Architecture. Objects identified by **IOR** (Interoperable Object Reference). Interface defined in **IDL** (language-neutral). **ORB** routes invocations. Supports both **static** (compiled stubs) and **dynamic** (DII) invocation.



Publish-Subscribe

Producers **publish** to topics, consumers **subscribe** to topics. Fully **decoupled**: producer doesn't know consumers, consumer doesn't know producers, neither knows when the other is active.

Apache Kafka:

- **Topic:** append-only, totally-ordered log split into **Partitions** (unit of parallelism + ordering)
- Each partition has one **Leader** broker (handles reads/writes) + N **Followers** (replicas)
- **Consumer Group:** each partition assigned to exactly one consumer – enables parallel consumption
- **Offset:** consumer position in the log; consumers control their own offset (can replay)
- **Retention:** messages kept for configurable time regardless of consumption



MQTT: lightweight pub/sub for IoT/edge. 3 QoS levels: 0 (at-most-once), 1 (at-least-once), 2 (exactly-once). Tiny protocol header for constrained devices.

IP Multicast: IGMP manages local group membership (hosts ↔ local router). Multicast routing protocols (DVMRP, PIM Dense/Sparse, CBT) build distribution trees. **PIM Sparse:** explicit join via Rendezvous Point (RP) – scalable for sparse groups.



3. CLOUD INFRASTRUCTURE

3.1. Kubernetes

Kubernetes (k8s)

Open-source container orchestration system. Declarative model: you describe the desired state (YAML manifest); controllers continuously reconcile actual state toward it.

Control Plane:

- **API Server** – single entry point; all components talk here
- **etcd** – only stateful component; distributed K-V store for all cluster state
- **Scheduler** – assigns pods to nodes based on resources/constraints
- **Controller Manager** – runs reconciliation control loops

Worker Node:

- **kubelet** – node agent; watches API server, executes PodSpecs
- **kube-proxy** – network proxy maintaining Service rules (iptables/IPVS)
- Container runtime (containerd / CRI-O)

Controller

pattern:

observe desired state → compare to actual → take action to reconcile

Workload Resource	Purpose
Pod	Atomic unit: one or more containers sharing network namespace and storage
ReplicaSet	Ensures N pod replicas running at all times
Deployment	ReplicaSet + rolling updates + rollback capability
StatefulSet	Stable network identity + persistent storage per pod (databases)
DaemonSet	One pod per node (logging agents, monitoring, CNI plugins)
Job / CronJob	Run-to-completion / scheduled batch tasks

Networking: **Service** (stable ClusterIP/NodePort/LoadBalancer endpoint) · **Ingress** (HTTP/HTTPS routing from outside, with TLS termination) · **CNI** plugins (Flannel, Calico, Cilium) implement pod networking.

RAFT Consensus (etcd)

Leader handles all writes; followers replicate. Majority quorum ($\frac{N}{2} + 1$) needed for any commit. Leader election on timeout. Guarantees linearizability for etcd reads (with `--consistency=linearizable`).

Autoscaling: **HPA** scales pod replicas based on CPU/memory/custom metrics. **KEDA** extends to event-driven sources (Kafka lag, SQS depth, cron) and enables **scale-to-zero**.

Service Mesh (Istio): sidecar proxy (Envoy) injected beside each pod. Provides: **Traffic management** (canary, circuit breaking, retries), **Observability** (traces, metrics, logs), **Security** (mTLS, authorization policies). Control plane: Istiod (Pilot + Citadel + Galley).



3.2. OpenStack

Service	Function	Project
Nova	Compute: provision and manage VM instances via hypervisors	compute
Neutron	Networking: virtual networks, subnets, routers, security groups	network
Cinder	Block storage: persistent volumes attached to VMs	volume
Swift	Object storage: distributed REST-accessible, replicated	object-store
Glance	Image service: disk images for VM boot	image
Keystone	Identity: authentication, token-based authZ, service catalog	identity
Horizon	Web dashboard for all services	dashboard
Ceilometer	Telemetry: metrics collection and alarms	telemetry
Heat	Orchestration: IaC using HOT templates	orchestration

VM Provisioning: User → Keystone (auth + token) → Nova API → Nova Scheduler (node selection) → Nova Compute → Neutron (attach network) → Cinder (attach volume) → Glance (pull image) → Hypervisor (boot).

Keystone 4 sub-services: Identity (users/groups) · Resource (projects/domains) · Assignment (role bindings) · Catalog (service endpoints).

3.3. Serverless & FaaS



FaaS – Function as a Service

Event-centric: stateless functions triggered by events, dynamically instantiated, billed per invocation. Provider manages everything below the function code. No persistent server process.



Cold Start

When a function is scaled to zero, the next request must provision a new container: pull image → start runtime → init function. Adds latency. **Mitigations:** slim images, pre-warmed pools, HTTP-mode watchdogs, provisioned concurrency.

Function Composition Patterns:

- **Continuous passing:** A → B → C → ... (pipeline)
- **Merging:** parallel branches → one function
- **Reflective:** function invokes next function
- **Façade:** single entry → fan-out to multiple
- **Map-Reduce:** split → parallel map → aggregate

BaaS – Backend as a Service: plug-and-play managed backend APIs (DB, auth, storage, CDN, AI) – no backend code written.

Platform	Architecture
OpenFaaS	k8s + Docker; Watchdog bridges HTTP→function
OpenWhisk	Kafka (triggers) + CouchDB (state)
Knative	k8s-native; Serving + Eventing
AWS Lambda	Fully managed, Firecracker micro-VMs

Knative Serving: Route → Configuration → Revision (immutable snapshot of code+config).



Autoscaler scales on RPS; Activator buffers when at zero.

Knative Eventing: **Broker+Trigger** for content-based routing; **Channel+Subscription** for fan-out. **CloudEvents** standard envelope.

3.4. Stream Processing

Spark Streaming (micro-batch):

- **DStream** = sequence of RDDs over time windows
- Processes one mini-batch at a time (1-5 s)
- Fault tolerance: RDD lineage + Write-Ahead Log
- Unified API with batch Spark
- Simpler model, slightly higher latency

Apache Flink (true streaming):

- Processes records one-by-one continuously
- Stateful operators; **distributed snapshots** (Chandy-Lamport barriers) for exactly-once
- Sub-second latency; native backpressure
- **JobManager** (coordination) + **TaskManagers** (execution, task slots)
- Pipelining: operators push data forward immediately

Processing guarantees: At-most-once (drop on failure) → At-least-once (replay, duplicates possible) → **Exactly-once** (requires idempotent state or 2PC – most expensive).



4. DISTRIBUTED COORDINATION

4.1. Replication for Dependability

Fault / Error / Failure

Fault: event that can cause problems. **Error**: incorrect internal state resulting from a fault. **Failure**: visible incorrect behavior observed externally. Goal: break the fault → failure chain.

- **Availability**: fraction of time the system is operational = $\frac{MTBF}{MTBF + MTTR}$
- **Reliability**: delivers only **correct** results (no wrong outputs – no errors leak out)
- **Recoverability**: can restore correct service state after a failure

RAID: RAID-0 = striping (perf only) · RAID-1 = mirroring (full copy) · RAID-5 = striping + distributed parity (1 disk fault) · RAID-6 = 2-disk fault tolerance.

TANDEM: special-purpose fault-tolerant system for continuous operation. Dual processors + dual paths; hardware-level fault isolation.

Passive Replication (Master-Slave)

Only the master executes. Slaves are standbys updated via **checkpoints**. On master failure, a slave takes over (failover). Checkpoint timing is critical: too frequent = overhead; too rare = lost work.

Active Replication

All copies execute all operations in the same total order (requires atomic multicast). No standby promotion needed. Higher coordination cost.

Lazy (async): fast writes, eventual consistency.
Eager (sync): consistent but higher latency.

5 Phases of Replication: ①Client Request → ②Copy Coordination → ③Copy Execution → ④Copy Agreement → ⑤Result Delivery

Update policy	Eager (sync)	Lazy (async)
Optimistic	Low inconsistency; possible rollback needed	High inconsistency; BASE / Eventual Consistency
Pessimistic	Strong consistency; high latency (2PC locks)	Inconsistency window; simpler recovery

ZooKeeper: Distributed coordination service. Stores small data (config, locks, leader info) in hierarchical **znodes**. Writes go through elected leader (majority ack). Reads served locally (may be stale). Sequential writes guarantee total ordering. Used by Kafka (pre-KRaft), HBase.

Docker Swarm: native k8s-lite orchestration in Docker. Manager nodes (RAFT consensus) + worker nodes. Services = desired state; tasks = running containers.



4.2 Group Communication & Multicast Ordering

Ordering	Guarantee	Cost
None	Members may see messages in any order	Minimum
FIFO	Same sender's messages arrive in send order at all receivers. Multi-sender interleaving is free.	Low
Causal	If m_1 causally precedes m_2 (Lamport $\Rightarrow$), then all members deliver m_1 before m_2 .	Medium
Atomic (Total)	All members deliver all messages in the same total order – regardless of sender.	High

Reliable Multicast: hold-back + NAK (negative-ack only on loss). **CATOCs / ISIS:** ABCast (atomic, cost $3(N-1)$) · CBCast (causal, partial order) · GBCast (group membership changes).

4.3 Synchronization & Logical Clocks

NTP – Network Time Protocol

Stratum hierarchy: 0=atomic clocks/GPS · 1=direct servers · N=cascaded (accuracy degrades). Corrects for network delay with 4 timestamps: T_1 (client sends), T_2 (server receives), T_3 (server sends), T_4 (client receives). Offset = $\frac{(T_2-T_1)+(T_3-T_4)}{2}$. Applies corrections by **slewing** (gradual rate adjust, no jumps).

Happened-Before (#so)

Partial order: ① a before b in same process $\rightarrow a \Rightarrow b$. ② send event $\Rightarrow$ receive event. ③ Transitivity. Events with no $\Rightarrow$ relation are **concurrent** ($\parallel$).

Lamport Logical Clock LC

Clock Condition: $a \Rightarrow b \rightarrow LC(a) < LC(b)$. **Not** bidirectional: $LC(a) < LC(b)$ does NOT imply $a \Rightarrow b$.

Rules: C1: $a \Rightarrow b$ same process $\rightarrow LC_{i(a)} < LC_{i(b)}$. C2: send $\Rightarrow$ recv $\rightarrow LC_{i(\text{send})} < LC_{j(\text{recv})}$.

Implementation: I1: increment LC_i on every event. I2: attach $TS = LC_{i(a)}$ to every sent message. I3: on receive: $LC_j = \max(TS_{\text{rcv}}, LC_j) + 1$.

Total order (break ties by pid): $a \Rightarrow b$ iff $LC_{i(a)} < LC_{j(b)}$, or $LC_{i(a)} = LC_{j(b)}$ and $P_i < P_j$.

Vector Clock $V[k]$

Each P_i maintains vector $V_i[k]$ for all processes. **Send:** $V_i[i] = V_i[i] + 1$, attach full vector. **Receive:** $V_j[k] = \max(V_j[k], V_i[k])$ for all k , then $V_j[j] = V_j[j] + 1$.



Bidirectional: $V_a < V_b$ iff $a \Rightarrow b$. Concurrent events $\parallel$ iff neither $V_a < V_b$ nor $V_b < V_a$. This is what Lamport clocks cannot detect.

4.4. Mutual Exclusion Algorithms

Algorithm	Messages/CS	Notes
Centralized Coordinator	3 (request + reply + release)	Simple; single point of failure; coordinator can be unfair
Lamport	$3(N - 1)$ or $N - 1 + 2$ bcasts	Decentralized; FIFO channels; no faults; static group; queue per process sorted by (ts, pid)
Ricart-Agrawala	$2(N - 1)$	Optimized Lamport: delay reply instead of queuing; no release message needed
Token Ring	N per round	Proactive (circulates even idle); simple single-fault recovery; token loss requires regeneration

Lamport Protocol: P_i sends $\text{REQUEST}(T_m, P_i)$ to all (including itself) $\rightarrow$ all reply $\rightarrow P_i$ enters CS when: (1) its request is first in local queue AND (2) reply received from every other process $\rightarrow$ sends RELEASE to all.

Ricart-Agrawala: on REQUEST from P_j , P_i **immediately replies** if not using CS or if P_j has higher priority (lower timestamp or lower pid); else **delays reply**. P_i enters when it has $N - 1$ replies.

4.5. Election Protocols

Bully Algorithm:

- P_i sends **Election** to all processes with higher ID
- If no reply within timeout $\rightarrow P_i$ wins, broadcasts **IAmCoordinator**
- Higher-ID process answers $\rightarrow$ stops P_i , starts its own election

$\rightarrow$ Highest alive process always wins

Ring Election:

- On timeout, P_i creates Election Token (ET) with its ID
- ET passed around ring; each node appends its ID
- Originator receives back its own ET $\rightarrow$ selects node with minimum index from the list
- That node generates the new coordination token

4.6. Global State & Distributed Snapshot



Consistent Cut



A cut (partition of events into “before” and “after”) is **consistent** if: whenever event e is in the cut, every event e' with $e' \Rightarrow e$ is also in the cut. Equivalently: no message is received without being sent. Prevents ghost messages and duplications.



Chandy-Lamport Snapshot Algorithm

Requires FIFO channels. **Marker** messages propagate the snapshot wave:

1. Initiator saves local state; sends **marker** on all OUT channels
2. First marker on IN channel C : save local state (if not done); start recording all subsequent messages arriving on all other IN channels; forward marker on all OUT channels
3. Subsequent markers on C : **close** recording on C – save buffered in-transit messages as channel state
4. All IN channels received marker $\rightarrow$ local snapshot complete

Global snapshot = union of all local process states + all channel states. Consistent by construction (FIFO + markers precede any post-snapshot message).



5. NETWORK QUALITY OF SERVICE

5.1. QoS Indicators & Management

Key QoS Indicators:

- **Bandwidth**: sustained data rate (bit/s)
- **Latency (RTT)**: round-trip time = $2 \times \frac{d}{v} + 2 \times \text{proc}$
- **Jitter**: variance of latency in a stream
- **Skew**: offset between multiple related flows
- **Loss rate**: % of packets dropped
- **QoE**: user-perceived quality (subjective)

Application types:

- **Elastic**: adapts to available BW (HTTP, FTP, email) – best-effort sufficient
- **Non-elastic / Real-time**: strict latency/jitter constraints (VoIP, live video, gaming) – need explicit QoS guarantees

SLA Lifecycle: Negotiation (agree on parameters) → Monitoring (verify adherence) → Enforcement (penalties or renegotiation on violation).

5.2. IntServ vs DiffServ



IntServ (per-flow)

RSVP two-phase signaling:

- **PATH**: sender → receiver (maps data path)
- **RESV**: receiver → sender (reserves resources)
- **Soft state**: reservations refresh periodically (no refresh = release)
- Per-flow state in **every** router → **does not scale** to Internet core
- **RTP**: in-band flow management messages
- **RTCP**: bidirectional control companion to RTP



DiffServ (per-class)

DS byte (DSCP) marks each packet at the edge:

- **EF (Expedited Forwarding)**: strict priority queue – low delay, low jitter (VoIP)
- **AF (Assured Forwarding)**: multiple classes with drop precedence (1/2/3)
- **BE (Best Effort)**: no guarantee

Traffic Conditioning at edge: Meter → Marker → Dropper/Shaper.

Scales: core routers only inspect DSCP, no per-flow state.

IntServ+DiffServ together: RSVP at edge for per-flow admission; DiffServ in core for class-based forwarding. Best of both.

5.3. Traffic Shaping & Scheduling

Leaky Bucket: output at **constant rate** r . Bursty input is buffered and smoothed. Excess dropped. Enforces strict average rate.

Token Bucket: tokens accumulate at rate r , bucket size b . Burst allowed up to b tokens. Short bursts at line rate; sustained rate = r . More flexible than leaky bucket.

Scheduling Policies:

- **FIFO**: simple, no fairness
- **Priority Queuing**: high-priority always first – starvation risk
- **Round Robin (WRR)**: weighted service per queue – fair
- **Max-Min Fairness**: satisfy smallest requests first; distribute surplus



- **Fair Queuing (GPS):** bit-by-bit fairness per flow, per-packet approximation

RED (Random Early Detection): probabilistic drop as queue fills – prevents synchronization, proactive congestion control.

SIP – Session Initiation Protocol: defines and manages multimedia sessions. Entities: UA (client), UAS (server), Proxy, Redirect, Registrar. Setup: INVITE → 100 Trying → 180 Ringing → 200 OK → ACK. Teardown: BYE.

SNMP: one manager + agents on managed devices. **MIB** defines object hierarchy. Versions: v1 (community string), v2c, v3 (auth + encryption). **RMON:** remote monitoring probes for traffic statistics.



6. BIG DATA INFRASTRUCTURE

6.1. Overlay Networks & Distributed File Systems



Chord DHT

Keys and nodes mapped to a **ring** modulo 2^m via SHA-1. Key k stored at **successor(k)**. **Finger table**: $\text{finger}[j] = \text{successor}(n + 2^{j-1})$ – achieves $O(\log N)$ lookup hops. **Consistent hashing**: node join/leave affects only immediate neighbors. Replication across r successors for fault tolerance.

Pastry: prefix-based routing. Node IDs and keys share common prefix – each hop shares one more prefix digit. $O(\log N)$ hops. Leaf set for local key ownership. Used in FreePastry, PAST.

GFS – Google File System:

- **Single master** (+ shadow replicas): holds only metadata; clients get chunk locations then bypass master for data
- **Large chunks** (64 MB): reduces master metadata load; favors streaming
- **Atomic record append**: GFS picks the offset; guarantees at-least-once per-record (not at-user-specified offset)
- **Mutations via leases**: primary chunkserver serializes concurrent writes within a lease period
- **Consistency**: after mutation – defined+consistent (serial), defined but inconsistent (concurrent success), undefined (failed mutations)
- **Replication**: 3 replicas (data flow pipelined through chain: client → R1 → R2 → R3)

HDFS – Hadoop Distributed File System: inspired by GFS. **NameNode** (single master, metadata in RAM) + **DataNodes** (chunk servers, 128 MB blocks). Replication pipeline; rack-aware placement (2 replicas same rack + 1 remote).

6.2. NoSQL: Cassandra & MongoDB



CAP Position of Key Systems

- **AP** (available + partition-tolerant): Cassandra, DynamoDB, CouchDB – eventual consistency
- **CP** (consistent + partition-tolerant): HBase, MongoDB (primary reads), ZooKeeper
- **CA** is impossible in a truly distributed system with network partitions

Apache Cassandra (AP, wide-column key-value store):

Architecture: Peer-to-peer ring; consistent **Bloom Filter**: compact bit array. On insert, set hashing maps keys to nodes. No single master. bits at k hash positions. On query, check all **Snitch** determines topology (DC/rack). **Gossip** k positions – **no false negatives**, small false positive rate. Avoids unnecessary SSTable disk reads.

Write path: Commit Log (WAL, durability) → Memtable (in-memory write) → SSTable (immutable)



on-disk flush). **Compaction** merges SSTables + removes tombstones. **Quorum:** $W + R > N \rightarrow$ strong consistency. $W = R = \text{QUORUM}$ (majority) is common.

Read path: check Memtable $\rightarrow$ Bloom filter (SSTable presence) $\rightarrow$ SSTable row cache / disk. **Consistency levels:** ONE $\cdot$ TWO $\cdot$ QUORUM $\cdot$ LOCAL_QUORUM $\cdot$ EACH_QUORUM $\cdot$ ALL.

Tunable: user picks R and W per operation – explicit consistency vs. availability trade-off.

MongoDB (CP, document store): BSON documents in collections. **Replica Set:** 1 primary + N secondaries; automatic failover via election (needs odd N). **Sharding:** shard key $\rightarrow$ mongos router $\rightarrow$ config server $\rightarrow$ shard. **Write concern:** 0/1/majority. **Read preference:** primary / secondary / nearest.

6.3. MapReduce, Hadoop YARN & Apache Spark



MapReduce

Programming model: **Map** function applied to each key-value pair $\rightarrow$ intermediate (k, v) pairs sorted and shuffled by key $\rightarrow$ **Reduce** aggregates values per key. Master assigns map and reduce tasks to workers. **Data locality:** prefer worker with local data replica.

Fault tolerance: re-execute failed map tasks (output on local disk, lost on failure). Reduce tasks re-execute on another node. **Backup tasks** (speculative execution) eliminate **stragglers** (slow workers).

YARN (Hadoop 2+): decouples resource management from job scheduling.

- **Resource Manager:** cluster master; allocates container resources
- **Node Manager:** per-node agent; manages containers
- **Application Master:** per-job coordinator; negotiates resources with RM



Spark RDD – Resilient Distributed Dataset

Distributed, immutable, in-memory collection partitioned across workers. **Transformations** (map, filter, join, groupBy) are **lazy** – build a **lineage graph** (DAG). **Actions** (collect, count, save) trigger execution. **Fault tolerance:** recompute lost partitions from lineage. **Persistence:** `cache()` / `persist()` to keep RDDs in memory across iterations.

Why Spark beats MapReduce on iterative workloads: RDDs stay in memory between iterations. MapReduce writes to HDFS after every map/reduce pair – 10-100× slower on iterative ML algorithms.

Spark Architecture: Driver (SparkContext, DAG scheduler) $\rightarrow$ Cluster Manager (YARN / k8s / Standalone) $\rightarrow$ Executors (task threads + block manager / cache on worker nodes).

Dimension	Spark Streaming	Apache Flink
Model	Micro-batching (DStreams = RDD windows)	True streaming (record-at-a-time)
Latency	~1-5 seconds	Sub-second (milliseconds)



Dimension	Spark Streaming	Apache Flink
State management	Via RDD lineage + WAL checkpoints	Managed stateful operators + key groups
Exactly-once	Yes, with WAL + idempotent sinks	Yes, via distributed snapshot barriers
Backpressure	Limited (rate control)	Native (credit-based)
Unified batch+stream	Yes (same Spark API)	Yes (DataStream $\approx$ DataSet API)
Windowing	Time-based DStream windows	Tumbling / Sliding / Session / Global